Name :- Aaryan N. Raorane Roll no :- 5004 branch :- AI & DS
batch :- A1 SAPID :- 60018250070

Experiment No 2

Title :- Decision - making and control statements

Aim :- To Implement python programs to demonstrate different decision making and control statements.

Theory :-

1. What is the difference between if, if.... else, and if... elf in Python? Give Examples.

⇒ **if statement :-**

The if statement is the simplest form of decision making in python. The block of code inside it executes only when the given condition evaluates to True. If the condition is False, the block is skipped Entirely and the program moves on.

code :-
```
marks = 75
if marks >= 40:
    print("student has passed")

# output : student has passed.
```

if.... else statement :-

The if.... else construct provides exactly two execution paths. If the condition is True, the if block runs. It is

Sundaram
FOR EDUCATIONAL USE

false, the else block runs. One of the two block is guaranteed to execute.

code:
```
marks = 30

if marks >= 40:
    print("Student has passed")
else:
    print("student has failed")

# output: student has failed.
```

if.... elif..... else statement

when there are more than two possible condition to check, the elif (short for 'else if') clause is used. Python evaluates each condition from top to bottom and executes the first block whose condition is True all remaining conditions are skipped. The else block at the end acts as a default case when no condition above it is satisfied.

code:
```
marks = 82
if marks >= 90:
    print("grade: O")
elif marks >= 75:
    print("grade: A")
elif marks >= 60:
    print("grade: B")
```

Sundaram
FOR EDUCATIONAL USE

```
elif marks >= 40:
    print("Grade: C")
else:
    print("Grade: F")

# output: Grade: A
```

The Key difference among the three is the number of conditions handled. if handles one condition, if...else handles two mutually exclusive outcomes, and if.... elif .... else handles multiple conditions in a single, clean structure.

2. Explain nested if statements with a real-life example.

=> A nested if statement is an if block placed inside another if block. Python first evaluates the outer condition. only if the outer condition is true does it enter that block and then evaluate the inner condition. This allows multiple levels of decision-making to be chained together.

Syntax:
```
if condition1:
    if condition2:
        # executes only when both condition1 and 2 are
          True
    else:
        # executes when condition1 is true but condition2
          is False
else:
    # Executes when condition1 is false
```

Sundaram
FOR EDUCATIONAL USE

Example code:

"""A bank approves a loan only if the applicant is above 18 years of age AND has a credit score above 700. If the applicant is underage, the loan is rejected outright without even checking the credit score"""

```
age = 22
credit_score = 750

if age >= 18:
    if credit_score >= 700:
        print("Loan approved.")
    else:
        print("Loan rejected: Low credit score.")
else:
    print("Loan rejected: Applicant is underage")
```

If age = 15, the output would be "Loan rejected: Applicant is underage" without the inner condition ever being checked. This is exactly how nested conditionals behave; the inner block is entirely unreachable when the outer condition fails.

Sundaram®
FOR EDUCATIONAL USE

3. How does Python determine equality between two strings?

→ Python compares two strings using the == operator, which performs a character-by-character comparison based on Unicode code points. Two strings are considered equal if and only if they have the same length and every character at each corresponding index is identical. The comparison is case-sensitive, meaning "Python" and "python" are not equal.

code:
```
s1 = "Python"
s2 = "Python"
s3 = "python"

print(s1==s2)  #output: True
print(s1==s3)  #output: False.
```

The == operator checks value equality (same content), while is checks identity (whether both variables point to the same object in memory). Two string variables can hold identical content but still fail an is comparison if they are stored at different memory locations.

code:
```
a = "Hello"
b = a
c = "Hello"
print(a==b)  # True
print(a is b)  # True
print(a is c)  # False
```

Sundaram® FOR EDUCATIONAL USE

4. Why is Indentation important in Python conditional statements? What happens if it is incorrect?

→ Unlike languages such as C, C++, and Java which use curly braces {} to define code blocks, Python uses indentation (whitespaces at the beginning of a line) to indicate which statements belong to a particular block. The Python interpreter treats indentation as part of the language syntax, not just a formatting convention.

All statements inside the same block must be indented by the same ~~block must~~ number of spaces. The standard convention is 4 spaces per indentation level.

Code:

```
x = 10
if x > 5:
    print("x is greater than 5")
    print("This also belongs to the if block")
print("This is outside the if block")

# output:
# x is greater than 5
# This also belongs to the if block
# This is outside the if block
```

If Indentation is missing or inconsistent, Python raises an IndentationError at parse time and the program does not run at all

Sundaram
FOR EDUCATIONAL USE

code: if a>5:
print("missing indentation")
IndentationError: expected an indented block

d More dangerous situation is logically incorrect indentation, where the code runs without error but produces wrong output. This is harder to catch.

code: a = 3

if a>5:
print("a is greater than 5")
print("this should be inside but is not")

The second print runs regardless of the condition.
output: This should be inside but is not.

In this case, the second print was intended to be part of the if block but was placed at the outer level by mistake, causing it to execute unconditionally. No error is raised, making such bugs difficult to detect.

Sundaram
FOR EDUCATIONAL USE

5. Differentiate between string concatenation and repetition in Python with examples.

→ String concatenation :-

Concatenation is the process of joining two or more strings end to end into a single string. In Python, this is done using the + operator. Both operands must be of type str; concatenating a string with an integer directly raises a TypeError.

code :-
```
first = "Data"
second = "Science"

result = first + " " + second
print (result) # output: Data Science
print(type(result)) # output: <class 'str'>

Date = "30th" + " " + "July"
print (Date) # output: 30th July.
```

If one operand is not a string, it must be explicitly converted using str() before concatenation

code :-
```
course = "Semester :"
num = 3
print(course + str(num)) # output: Semester: 3
```

Sundaram®
FOR EDUCATIONAL USE

String Repetition :-

Repetition produces a new string by repeating the original string a specified number of times. In Python, this is done using the * operator between a string and an integer

code :-
```
word = "Ha"
print(word * 3)  # output: HaHaHa

separator = "=" * 20
print(separator) #output: ====================

line="Python!"
print(line * 2)  #output: Python! Python!
```

Key difference :-

Concatenation (+) joins distinct strings together, while repetition (*) multiplies a single string. Concatenation requires atleast two string operands, whereas repetition requires one string and one integer operand. Mixing up the two operations or their types will result in a TypeError at runtime.

Sundaram®
FOR EDUCATIONAL USE

Conclusion :-

Through this experiment, the decision-making constructs in Python namely if, if---else, if---elif--else and nested if statements were studied and implemented with examples. The role of indentation as a syntactic rule in Python was understood along with the errors caused by incorrect indentation. String equality using the == operator and the difference between string concatenation and repetition were also demonstrated practically.

Sundaram
FOR EDUCATIONAL USE